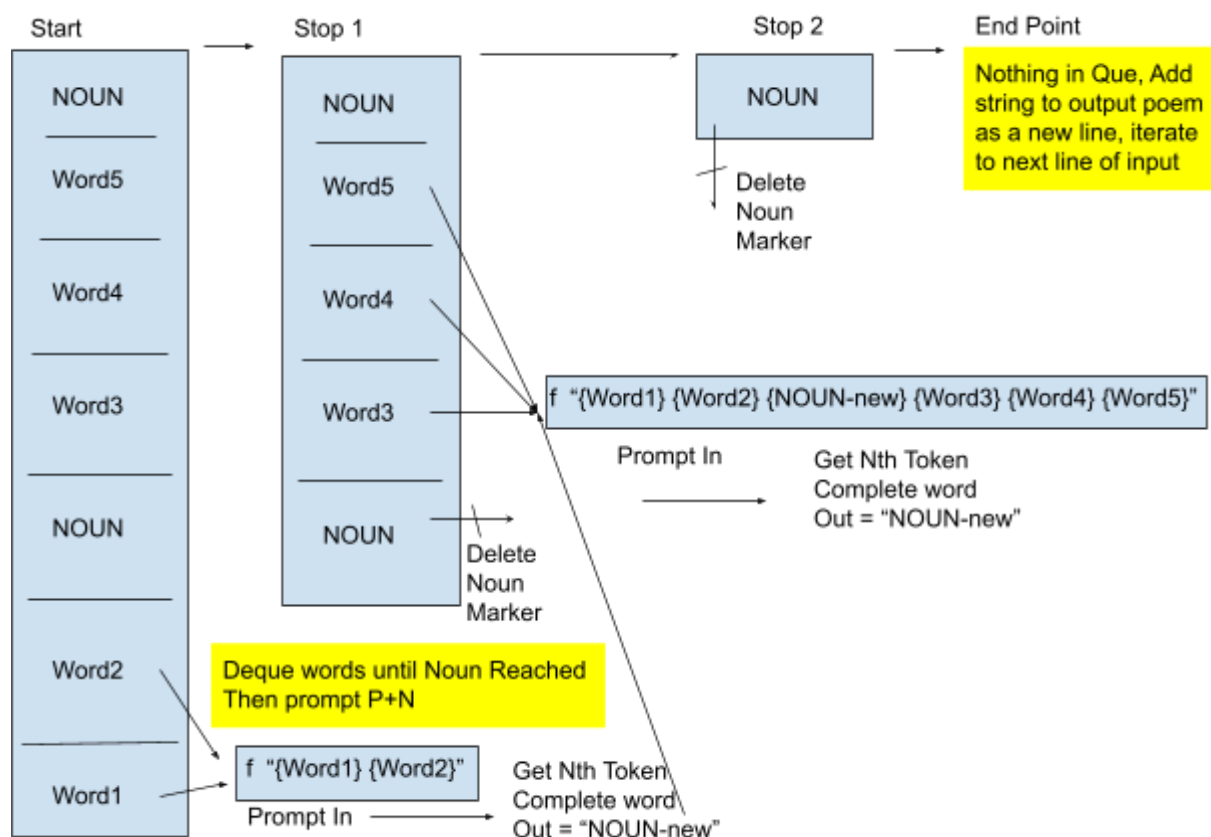


Changing the N value slowly drifted the sentiment of the final word off in different directions, gradually distorting the output from things that made sense to strange but coherent, followed by obscure words that would seem out of place. Roughly past the 20k mark, run on words, what I assume to be 'acronyms', run on words and other strange artifacts became a lot more noticeable, and around the 50k mark Japanese words became common throughout adjacent N values. 100 - 1000 was a sort of 'sweet spot' for funny things to generate. My chance to have the predictions 'finish' word fragments led to some funny at all stages, with uncommon starting tokens after $n = 20k$ the 'word finisher' had some weird effects on the output, with the output creating non-words generated from weird initial token on top of the finished non-words. The word finisher still only uses the most probable next token to end a word, changing that to the n th token would probably increase the weirdness of the output and is something I plan on playing with more.

The implementation for replacing all nouns can be approached in two ways, I'll go over the simpler one first. Assuming individual nouns are manually marked with a special character, or the special character replaces the entire noun. When each line is split by spaces into a list object, the processing would look something like what follows



The final output of this line would be `"{Word1} {Word2} {NOUN-new} {Word3} {Word4} {Word5} {NOUN-new}"`. The second approach would be to auto identify the nouns, if the same marker is used when identifying nouns, the previous system would work for the implementation.